

클라우드 인프라 교육 · 1일차 오전

클라우드 컴퓨팅 이해

개념 · 아키텍처 · 핵심 생태계 · 트렌드

제1장 클라우드 컴퓨팅 개요

제2장 클라우드 아키텍처 및 서비스 모델

제3장 클라우드 생태계의 핵심 개념

제4장 트렌드와 퍼블릭 클라우드의 가치·이슈

오늘 오전의 목표

- **전체 판 그리기** — 클라우드 생태계가 어떤 요소들로 이루어져 있고 서로 어떻게 연결되는지 지도를 그린다.
- **개념 명확화** — IaaS/PaaS/SaaS, 리전/AZ, SDN, 제로 트러스트, MSA, CI/CD 등 자주 듣지만 흐릿했던 용어를 정확히 잡는다.
- **올바른 구축 방향** — 시스템을 클라우드에 올릴 때 무엇을 기준으로 판단해야 하는지 원칙을 세운다.

제1장

클라우드 컴퓨팅 개요

정의 · 동작 원리 · 배포 모델 · 과금 구조 · 국내 규제

1.1 클라우드 컴퓨팅이란

**IT 자원을 직접 구매하지 않고,
인터넷을 통해 필요한 만큼 빌려 쓰고 쓴 만큼 지불하는 방식**

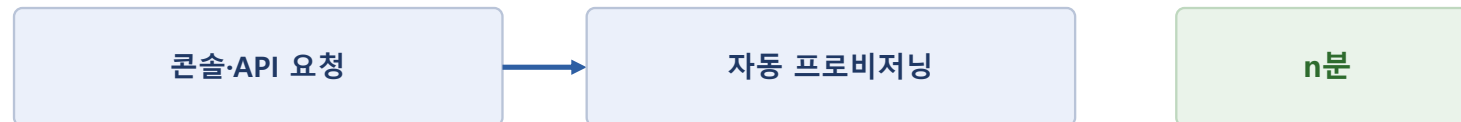
- 소유가 아닌 사용 — 하드웨어는 CSP가 소유, 사용자는 사용권만 보유
- 셀프서비스 — 웹 콘솔·API로 직접 자원을 생성·삭제 (품의·계약 불필요)
- 종량제 — 시간·용량·전송량이 초/GB 단위로 계측되어 과금

서버 한 대를 얻기까지 — 방식의 차이

온프레미스



클라우드



조달 리드타임의 소멸 — 이 속도 차이가 클라우드가 비즈니스를 바꾼 첫 번째 지점

클라우드는 실제로 어떻게 동작하는가

"서버 1대 생성" 요청 시 내부에서 벌어지는 일

1

사용자가 콘솔·CLI·API로 서버 생성을 요청 (예: EC2 t3.micro, Amazon Linux)

2

CSP 제어 시스템(Control Plane)이 여유 있는 물리 서버를 선택

3

하이퍼바이저가 요청 사양만큼 자원을 잘라 가상 머신 생성

4

OS 이미지(AMI) 복사, 가상 네트워크 카드에 IP 할당

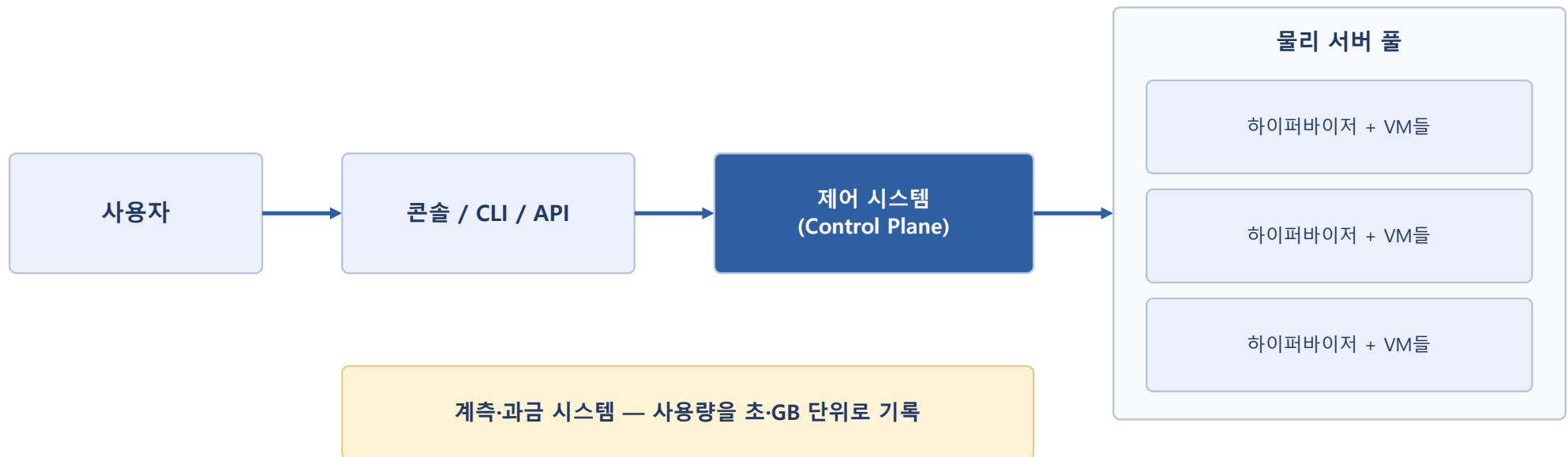
5

1~2분 후 접속 가능 — 이 순간부터 초 단위 과금 시작

6

종료 시 자원은 풀(Pool)로 반환되고 과금 중지

그림으로 보는 클라우드의 내부 구조



자원 풀 + 가상화 + 자동화된 제어 + 계측·과금 — 4개 부품의 결합이 클라우드의 전부다

클라우드의 실체

거대한 물리 서버 풀 + 가상화 기술
+ 자동화된 제어 시스템 + 계측·과금 체계

- 마법이 아니라 자동화의 극한이다
- 이 구조를 이해하면 이후 모든 개념(가상화·SDN·API)이 연결된다

클라우드의 5가지 핵심 특징

진짜 클라우드와 단순 호스팅을 구분하는 기준

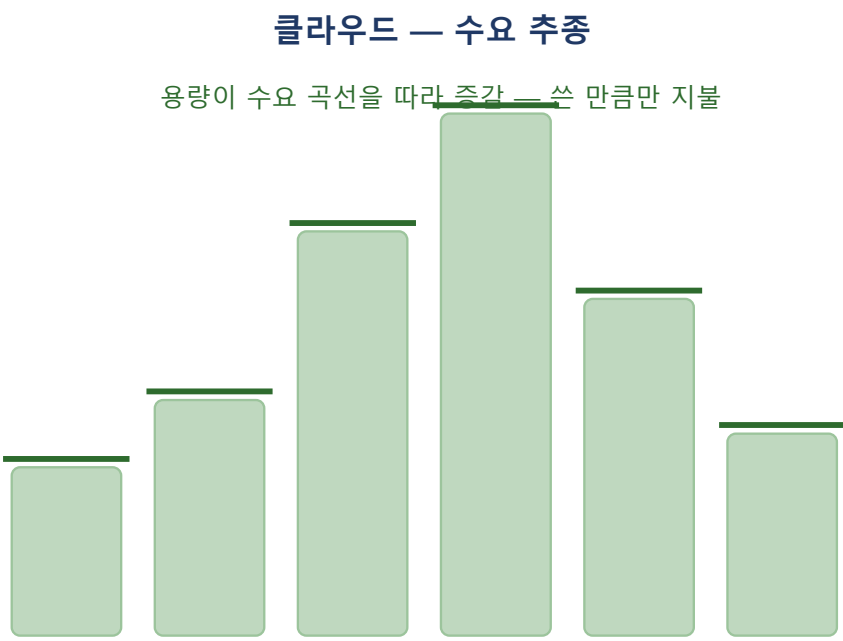
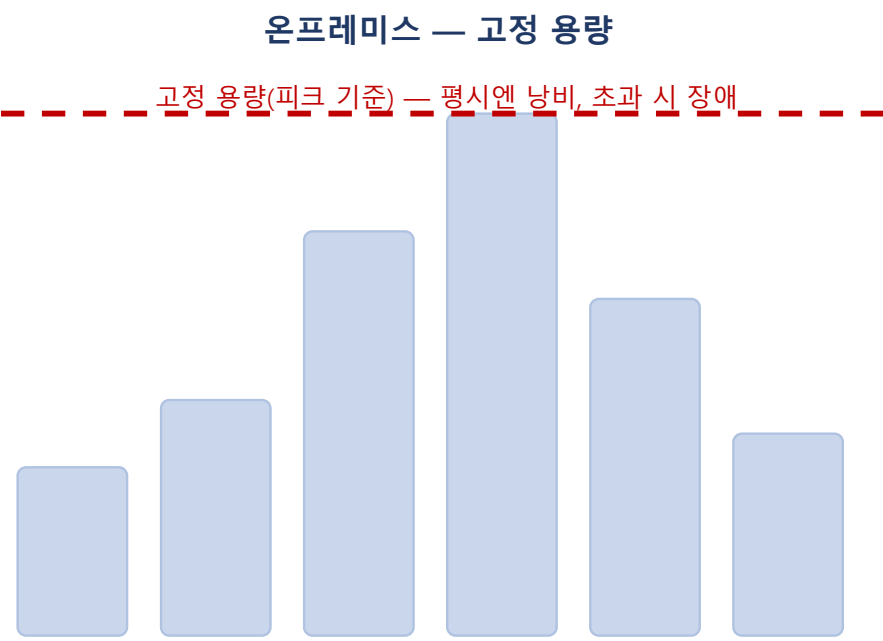
특징	의미	실무 예시
온디맨드 셀프서비스	사업자 개입 없이 직접 자원 생성·삭제	콘솔 클릭 몇 번으로 서버 생성
광범위한 네트워크 접근	표준 프로토콜로 어디서나 접근	사무실·집·모바일 어디서든
리소스 풀링	물리 자원을 다수 사용자가 논리적으로 분할 사용	한 물리 서버에 여러 고객 VM 격리 동작
신속한 탄력성	수요에 따라 즉시 확장·축소	이벤트 때 2대→20대, 끝나면 다시 2대
측정 가능한 서비스	사용량 계측 기반 투명 과금	청구서에 항목별 요금 표시

가장 중요한 특징 — 탄력성

탄력성이 없다면 클라우드를 쓸 이유의 절반이 사라진다

- 부하 변동이 큰 워크로드일수록 클라우드 이점이 커진다
- 반대로 부하가 일정한 워크로드는 이점이 상대적으로 작다 — 도입 판단의 첫 기준

탄력성을 그림으로 — 용량이 수요를 따라간다



막대 = 시간대별 수요, 선 = 확보한 용량 — 두 선의 간격이 곧 낭비(왼쪽)와 절감(오른쪽)이다

클라우드 등장 배경

- **서버 활용률 문제** — 온프레미스 서버는 피크 기준 용량 산정으로 평균 활용률 10~20%. 낭비를 리소스 풀링으로 해결
- **가상화 기술의 성숙** — 2000년대 초 x86 가상화 안정화. 물리 서버 1대를 수십 VM으로 분할 가능
- **인프라 조달 속도의 한계** — 발주~설치 수 주~수 개월. 분 단위 프로비저닝 수요 급증
- **규모의 경제** — 아마존·구글이 자체 초대형 인프라 운영 노하우와 잉여 용량을 외부 판매. AWS는 2006년 S3·EC2 출시

클라우드 발전 과정

시기	단계	핵심 내용
~1990년대	온프레미스 시대	기업별 자체 서버·전산실 구축이 표준
2000년대 초	가상화 확산	VMware 중심 서버 통합, 물리 서버 수 감소
2006년	IaaS 상용화	AWS EC2/S3 출시, 종량제 인프라 시장 개막
2010년대 초	PaaS·SaaS 확산	Heroku, Office 365 — 플랫폼·소프트웨어로 확장
2013~2015	컨테이너 혁명	Docker(2013), Kubernetes(2014), Lambda(2014)
2020년대	AI·멀티클라우드	GPU 수요 폭증, 멀티·하이브리드 보편화

1.2 온프레미스 vs 클라우드

비교 항목	온프레미스	클라우드
자산 소유	조직이 직접 소유 (자산·감가상각)	CSP 소유, 사용권만 구매
비용 구조	CAPEX(초기 투자) 중심	OPEX(운영비) 중심, 종량제
조달 기간	수 주 ~ 수 개월	수 분
용량 산정	피크 기준 사전 확정	수요 따라 실시간 조정
확장	물리 증설 (공간·전력 제약)	한도 내 사실상 무제한
보안 책임	전 계층 자체 책임	물리=CSP / 설정·데이터=사용자
재해복구	별도 DR센터 (고비용)	타 리전 복제로 저비용 구성

비용 비교 — 시나리오로 판단하기

"클라우드가 무조건 싸다"는 틀린 말이다

시나리오 A — 이벤트성 쇼핑몰

- 평시 2대, 월 1회 프로모션 3일간 20대 필요
- 온프레미스: 20대 상시 보유 → 18대가 한 달의 90% 유휴
- 클라우드: 평시 2대 + 이벤트 3일치 18대 요금만
- → 클라우드가 압도적으로 유리

시나리오 B — 상시 일정 부하 ERP

- 24시간 365일 동일 부하
- 탄력성 이점 없음
- 3~5년 장기 운영 시 감가상각 끝난 온프레미스가 저렴할 수 있음
- → 예약 인스턴스 또는 온프레미스 유지 검토

도입 판단의 원칙

**"클라우드냐 아니냐"의 이분법이 아니라
워크로드별 TCO(총소유비용) 비교로 판단한다**

- $TCO = \text{장비} + \text{전력} + \text{상면} + \text{인건비} + \text{유지보수의 3~5년 합산 비용}$
- 부하 변동성이 클수록, 시작 속도가 중요할수록 클라우드가 유리

클라우드의 이점과 리스크

이점

- 속도 — PoC를 당일 시작, 조달 병목 제거
- 탄력성 — Auto Scaling으로 과잉투자·장애 동시 방지
- 가용성 — Multi-AZ 분산으로 저비용 고가용성
- 글로벌 — 해외 리전 복제 배포
- 관리형 서비스 — DB·AI 등을 전문 인력 없이 사용

리스크

- 비용 폭주 — 끄지 않은 자원, 아웃바운드 트래픽
- 벤더 종속 — 전용 서비스 의존 시 이전 곤란
- 설정 실수 — 보안 사고 대부분은 사용자 설정 오류
- 규제 제약 — 공공·금융의 인증 요건
- 네트워크 의존 — 회선 장애 시 업무 중단

1.3 배포 모델 한눈에 — 누가 소유하고 누가 쓰는가



1.3 배포 모델 ① 퍼블릭 / 프라이빗

- **퍼블릭 클라우드** — CSP 인프라를 다수 고객이 공용 사용 (AWS·Azure·GCP). 계정 단위 논리적 완전 격리. 초기 투자 불필요·무제한 확장이 강점, 통제 제한·규제 요건이 약점
- **프라이빗 클라우드** — 단일 조직 전용. 주의: 가상화만 깔았다고 프라이빗 클라우드가 아니다. 셀프서비스·자동 프로비저닝·계측까지 구현해야 클라우드
- **프라이빗 구축 기술** — VMware, OpenStack, Red Hat OpenShift
- **프라이빗 적합처** — 금융 핵심계, 국방·기밀, 데이터 반출 불가 워크로드

1.3 배포 모델 ② 하이브리드 / 멀티클라우드

- **하이브리드** — 퍼블릭+프라이빗을 전용선(Direct Connect 등)으로 묶어 하나의 체계로 운영. 국내 대기업의 가장 흔한 형태
- **활용 패턴** — ① 민감 데이터는 내부, 웹·API는 퍼블릭 ② 클라우드 버스팅(피크 초과분만 퍼블릭) ③ 전환 과도기 아키텍처
- **멀티클라우드** — 복수 퍼블릭 CSP 병행. 종속 완화·강점 조합·장애 이중화가 목적
- **멀티의 비용** — CSP별 기술 스택 상이로 운영 복잡도 배증. K8s·Terraform 같은 중립 기술이 공통 언어 역할

하이브리드 클라우드의 구조

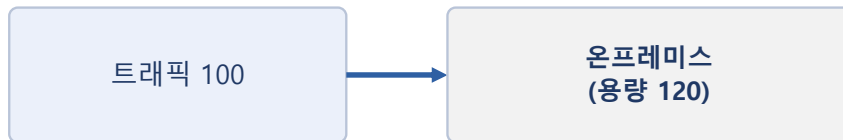
기업의 사용 형태



민감한 것은 안에, 변동이 큰 것은 밖에 — 두 환경을 전용 네트워크로 묶어 하나의 시스템처럼 운영

하이브리드 활용 패턴 — 클라우드 버스팅

평시 — 자체 인프라로 처리



이벤트 피크 — 초과분만 클라우드로

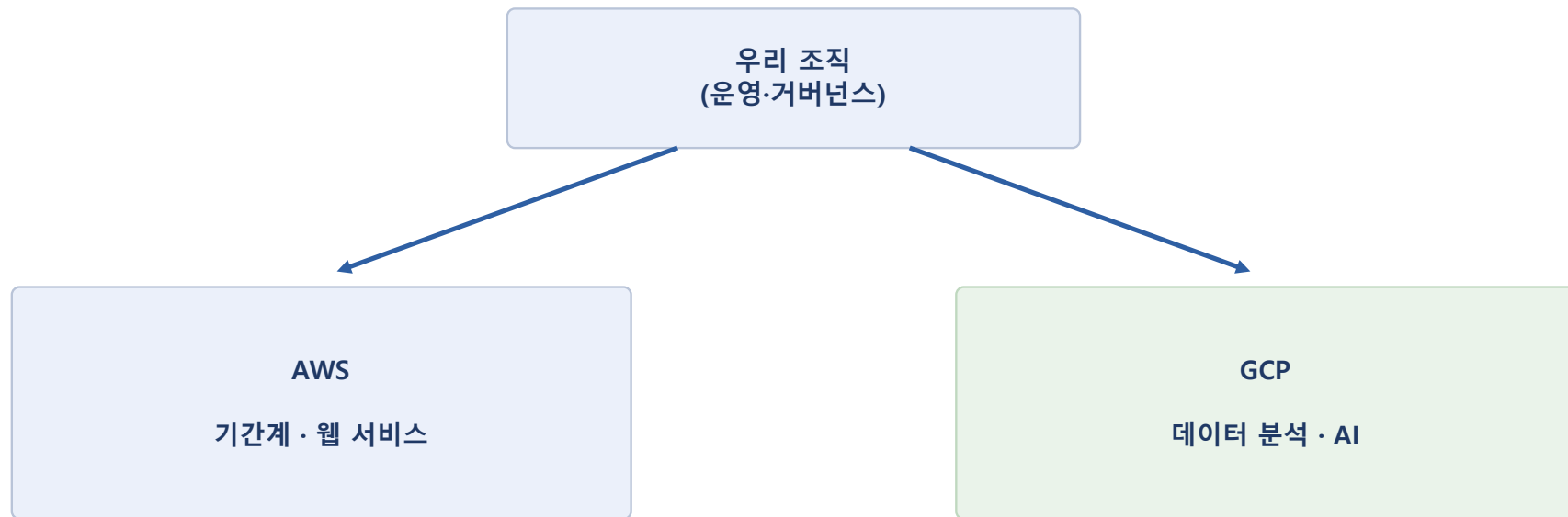


자체 인프라는 평시 수준만 보유 — 피크의 비싼 용량은 그때만 빌린다

하이브리드 설계 시 챙길 것들

- **네트워크 품질** — 전용선(Direct Connect)의 대역폭·지연이 전체 성능을 좌우. VPN은 백업 경로로 이중화
- **데이터 위치 원칙** — 어떤 데이터가 어느 쪽에 있어야 하는지 규제·성능 기준으로 명문화
- **보안 정책 일관성** — 두 환경의 계정·권한·방화벽 정책이 어긋나면 그 틈이 곧 보안 구멍
- **통합 관측** — 모니터링·로그를 한 화면으로 — 장애 시 "어느 쪽 문제인가"를 빨리 갈라야 한다
- **운영 숙련** — 두 환경 모두를 다루는 인력 필요 — 하이브리드의 숨은 비용

멀티클라우드의 구조



CSP별 강점에 워크로드를 배치하되, 중립 기술로 운영 방식을 통일하는 것이 성패의 관건

멀티클라우드 — 왜 하고, 무엇을 감수하나

- **채택 이유 ①** — 벤더 종속 완화: 특정 CSP 가격 인상·정책 변경에 대한 협상력 확보
- **채택 이유 ②** — 강점 조합: 기간계는 AWS, 분석·AI는 GCP처럼 잘하는 곳에 맡김
- **채택 이유 ③** — 가용성: 한 CSP의 대규모 장애 시에도 서비스 지속
- **감수할 비용** — 콘솔·기술 스택이 CSP마다 달라 운영 복잡도와 인력 요구가 배로 증가
- **현실적 전략** — 완전 이중화보다 "주력 CSP + 특정 워크로드 보조 CSP"가 일반적 출발점

커뮤니티 클라우드와 배포 모델 선택 가이드

- **커뮤니티 클라우드** — 같은 목적(규제·보안 요건)을 가진 복수 조직이 공동 사용. 국내 공공 전용 클라우드 존이 유사 사례
- **선택 가이드 ①** — 규제·기밀이 최우선 → 프라이빗 (또는 CSAP 인증 퍼블릭)
- **선택 가이드 ②** — 속도·확장이 최우선 → 퍼블릭
- **선택 가이드 ③** — 둘 다 필요 → 하이브리드로 분리 배치
- **선택 가이드 ④** — 종속 리스크가 크거나 CSP별 강점이 뚜렷 → 멀티 (복잡도 감수)

배포 모델 비교 정리

구분	퍼블릭	프라이빗	하이브리드	멀티
인프라 소유	CSP	자체	혼합	복수 CSP
초기 비용	낮음	높음	중간	낮음
통제 수준	제한	완전	부분	제한
운영 복잡도	낮음	높음	높음	매우 높음
핵심 목적	민첩성·확장	통제·규제	균형·전환	종속 회피

1.4 과금 구조(AWS) — 컴퓨팅 구매 옵션

옵션	할인율	특징	적합 대상
온디맨드	기준가	약정 없음, 초 단위 과금	단기·검증·예측 불가 부하
예약형/Savings Plans	~70%	1년/3년 약정	상시 가동 서버
스팟	~90%	유휴 자원 활용, 회수 가능성	배치·병렬 등 중단 허용 작업
프리티어	무료	신규 계정 12개월 한정	학습·검증

실무 표준: 기본 부하=예약형 + 변동 부하=온디맨드 + 배치=스팟 조합 → 컴퓨팅 비용 절반 수준까지 절감 가능

요금 (서울 리전 온디맨드, 자릿수 감각)

- **소형 서버 t3.micro** — 시간당 약 \$0.01~0.02, 한 달 상시 가동 시 약 1~2만 원대
- **중형 서버 m5.xlarge** — 시간당 약 \$0.2대, 한 달 상시 가동 시 수십만 원대
- **S3 표준 스토리지** — GB당 월 약 \$0.025, 100GB 저장에 월 수천 원
- **아웃바운드 트래픽** — GB당 약 \$0.1 내외, 월 10TB 반출이면 그것만 백만 원 이상
- **얻어야 할 직관** — 진짜 비용은 상시 가동 중대형 서버와 아웃바운드 트래픽에서 나온다. "안 쓰는 서버 끄기"와 "트래픽 설계"가 양대 축

놓치기 쉬운 과금 항목

- **중지한 인스턴스의 디스크** — EC2 중지(Stop) 시 컴퓨팅 요금은 멈추지만 EBS 볼륨 요금은 계속
- **연결 해제된 볼륨·미사용 고정 IP** — 인스턴스를 지워도 남아 있으면 과금
- **리전 간·AZ 간 전송** — 같은 AWS 내부여도 경계를 넘으면 요금 발생
- **요청 단위 과금** — S3 PUT/GET, DNS 질의 등 건수 기반 항목
- **스냅샷·백업 누적** — 자동 백업이 쌓이며 조용히 증가. 보존 기간 정책 필요

1.5 국내 시장 구조와 규제

- **글로벌 CSP** — AWS(서울 2016), Azure(한국 중부/남부), GCP(서울 2020)가 민간 시장 주도
- **국내 CSP** — 네이버클라우드·KT클라우드·NHN클라우드·삼성SDS. 공공·금융 특화, 인증 요건으로 공공 비중 높음
- **MSP** — 메가존클라우드·베스핀글로벌 등 도입 설계·구축·운영 대행 사업자군
- **CSAP** — 공공기관 공급 필수 국가 인증. 상·중·하 등급별 요건(물리/논리 분리) 차등
- **ISMS-P / 전자금융감독규정 / 개인정보보호법** — 클라우드 이용 환경이 심사 범위에 포함, 리전 선택에 직접 영향

제1장 점검

- ☐ 서버 생성 요청 시 클라우드 내부 동작을 순서대로 설명할 수 있는가?
- ☐ 탄력성이 왜 핵심인지, 어떤 워크로드에서 이점이 작은지 설명할 수 있는가?
- ☐ 온디맨드·예약형·스팟의 차이와 조합 전략을 말할 수 있는가?
- ☐ EC2를 중지해도 계속 과금되는 항목을 아는가?
- ☐ CSAP가 어떤 제도이고 누구에게 적용되는지 아는가?

제2장

클라우드 아키텍처 및 서비스 모델

IaaS/PaaS/SaaS · 리전과 AZ · EC2 · 스토리지 · VPC · 확장성 설계

2.1 서비스 모델의 본질

IaaS / PaaS / SaaS 구분의 본질은 기술이 아니라
"관리 책임의 경계"다

- 어디까지 CSP가 관리하고, 어디부터 우리가 관리하는가
- 선택에 따라 필요한 인력·운영 범위·장애 책임이 달라진다

IaaS (Infrastructure as a Service)

- **제공 범위** — 가상 서버·스토리지·네트워크 등 인프라 자원
- **사용자 책임** — OS 설치·패치, 미들웨어, 애플리케이션, 백업, 모니터링 전부
- **대표 서비스** — AWS EC2, Azure VM, GCP Compute Engine
- **적합 용도** — 온프레미스 이전(수정 최소화), 특수 미들웨어, 세밀한 튜닝 필요 시
- **특성** — 통제 범위 최대 = 운영 책임 최대

PaaS (Platform as a Service)

- **제공 범위** — OS·런타임·미들웨어까지 CSP 관리, 코드만 올리면 배포·확장 자동
- **사용자 책임** — 애플리케이션 코드 품질, 설정, 데이터
- **대표 서비스** — Elastic Beanstalk, Azure App Service, Heroku
- **적합 용도** — 신규 웹/API 개발, 인프라 인력 없는 팀, 빠른 출시
- **한계** — 플랫폼 지원 범위 밖의 커스터마이징 불가

SaaS (Software as a Service)

- **제공 범위** — 완성된 소프트웨어를 구독 사용. 설치·운영·업그레이드 전부 사업자 몫
- **사용자 책임** — 계정·권한 관리, 데이터 등급 분류와 반출 통제
- **대표 서비스** — Microsoft 365, Google Workspace, Salesforce, Slack
- **적합 용도** — 그룹웨어·협업·CRM 등 "직접 만들 이유가 없는" 범용 기능

그림으로 보는 관리 책임의 경계



관리 책임 경계표

주목: 데이터는 어느 모델에서도 사용자 책임 — 공동 책임 모델의 출발점

계층	온프레미스	IaaS	PaaS	SaaS
애플리케이션	사용자	사용자	사용자	CSP
데이터	사용자	사용자	사용자	사용자
런타임·미들웨어	사용자	사용자	CSP	CSP
운영체제	사용자	사용자	CSP	CSP
가상화	사용자	CSP	CSP	CSP
서버·스토리지·네트워크	사용자	CSP	CSP	CSP
데이터센터	사용자	CSP	CSP	CSP

파생 모델 — FaaS / CaaS / DBaaS

- **FaaS** — 함수 단위 코드를 이벤트 기반 실행, 밀리초 과금. 서버리스의 핵심 구현체 (AWS Lambda)
- **CaaS** — 컨테이너 실행 환경을 관리형 제공. 관리형 K8s(EKS·AKS·GKE). IaaS와 PaaS의 중간
- **DBaaS** — DB 설치·패치·백업·복제 대행 (AWS RDS)

2.2 글로벌 인프라 — 리전 / AZ / 엣지

- **리전(Region)** — 지리적으로 독립된 서비스 단위, 전 세계 30여 곳. 서울 = ap-northeast-2. 리전 간 기본 격리
- **가용영역(AZ)** — 리전 내 전력·네트워크·냉각이 분리된 데이터센터 그룹. 서울 4개(2a~2d). AZ 간 저지연 전용망
- **엣지 로케이션** — CDN·DNS용 소규모 거점, 전 세계 수백 곳. 사용자 인접 전달로 지연 최소화
- **리전 선택 기준** — ① 지연시간(사용자 근접) ② 규제·데이터 주권 ③ 서비스 가용성 ④ 요금

리전 · 가용영역 · 엣지의 관계



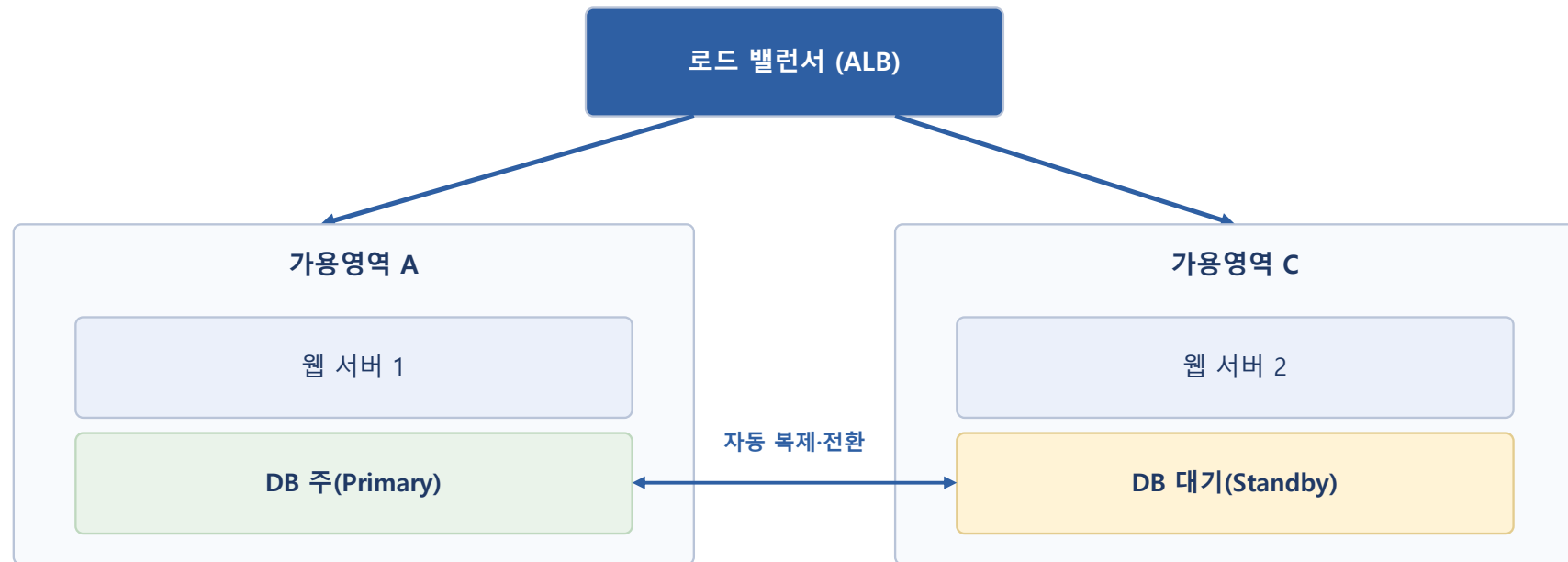
리전 = 지리적 단위, AZ = 리전 안의 물리적 분리, 엣지 = 사용자 앞 최전선 — 3계층 구조

Multi-AZ — 고가용성의 기본 공식

고가용성 제1원칙 = 단일 장애점(SPOF) 제거
클라우드의 기본 수단 = Multi-AZ 분산 배치

- 웹 서버를 AZ-a·AZ-c에 나눠 배치 + 로드 밸런서 → 한 AZ 정전에도 서비스 유지
- RDS Multi-AZ 옵션 → 대기 복제본 자동 유지, 장애 시 자동 전환(Failover)

Multi-AZ 구성의 실제 모습



한 AZ 전체가 정전돼도: ALB가 살아있는 AZ로만 트래픽 전달 + DB는 대기본으로 자동 전환(Failover)

2.3 EC2 — 인스턴스 타입 읽는 법

t3.micro = t(패밀리) + 3(세대) + micro(크기)

- **패밀리** — 용도 구분 문자. T/M(범용), C(컴퓨팅), R(메모리), I/D(스토리지), P/G(GPU)
- **세대** — 숫자가 클수록 신형. 신형이 대체로 가성비 우수
- **크기** — micro → small → medium → large → xlarge → 2xlarge... 단계마다 대략 2배

EC2 인스턴스 패밀리

패밀리	예시	특성	주 용도
범용 (T/M)	t3.micro, m5.large	CPU·메모리 균형, T는 버스트형	웹 서버, 개발·테스트
컴퓨팅 최적화 (C)	c5.xlarge	vCPU 비중 높음	고성능 연산, 게임 서버
메모리 최적화 (R/X)	r5.xlarge	메모리 비중 높음	인메모리 캐시, 대용량 DB
스토리지 최적화 (I/D)	i3.large	고속 로컬 디스크	NoSQL, DW
가속 컴퓨팅 (P/G)	p4d, g5	GPU 탑재	AI 학습·추론, 렌더링

인스턴스 생명주기와 과금

- **실행 중(Running)** — 컴퓨팅 요금 + EBS 요금
- **중지(Stopped)** — 컴퓨팅 요금 중단, EBS 요금 지속. 재시작 시 퍼블릭 IP 변경 가능
- **종료(Terminated)** — 완전 삭제. 기본 설정에서 루트 볼륨도 함께 삭제
- **AMI** — OS+설정이 담긴 서버 템플릿. 동일 서버 복제 생성의 기준점

2.4 스토리지 세 가지 유형

구분 기준은 접근 방식

유형	대표	접근 방식	주 용도
블록 스토리지	EBS(Elastic Block Storage)	인스턴스에 디스크로 장착	OS 부팅, DB 저장소
오브젝트 스토리지	S3(Simple Storage Service)	HTTP API로 객체 단위	이미지·백업, 정적 웹, 데이터 레이크
파일 스토리지	EFS(Elastic File System)	NFS로 다수 서버 동시 마운트	공유 파일 시스템

스토리지 3유형 — 연결 방식이 다르다



OS·DB 디스크는 EBS / 이미지·백업·로그는 S3 / 공유 폴더는 EFS — 용도로 기억

S3 스토리지 클래스

수명주기 정책과 결합해 비용 최적화

클래스	특성	용도
S3 Standard	즉시 접근, 최고 가용성	자주 접근하는 운영 데이터
Standard-IA	저장 단가 낮음, 조회 시 과금	접근 빈도 낮은 백업
Glacier 계열	초저가 장기 보관, 조회에 수 분~시간	법정 보존, 장기 백업
Intelligent-Tiering	접근 패턴 학습해 자동 이동	패턴 예측 어려운 데이터

2.6 확장 전략 — Scale-up vs Scale-out

수직 확장 (Scale-up)

- 서버 1대의 사양 상향 (t3.micro→m5.large)
- 구성 단순
- 물리적 상한 존재
- 단일 장애점(SPOF) 문제

수평 확장 (Scale-out)

- 같은 역할 서버 수 증가 + 로드 밸런서
- 상한 사실상 없음, 1대 장애에도 서비스 유지
- 클라우드 네이티브 표준
- 전제: 애플리케이션의 무상태(Stateless) 설계

상태(State) : 서버가 요청을 처리하면 기억해두는 정보
ex) 로그인 세션, 장바구니 내용, 사용자 업로드 파일

무상태(Stateless): 서버가 아무것도 기억하지 않는 설계
ex) 파일 -> S3, 데이터 -> DB

Auto Scaling의 동작

1

기준 지표 설정 (예: 그룹 평균 CPU 60%)

2

최소·희망·최대 대수 설정 (예: 2 / 2 / 10)

3

기준 초과 시 자동 추가, 부하 하락 시 자동 제거

4

헬스체크 실패 인스턴스는 자동 교체 — 자가 치유(Self-healing)

2.7 주요 CSP 서비스 대응표

기능	AWS	Azure	Google Cloud
가상 서버	EC2	Virtual Machines	Compute Engine
오브젝트 스토리지	S3	Blob Storage	Cloud Storage
가상 네트워크	VPC	VNet	VPC
관리형 RDB	RDS	Azure SQL	Cloud SQL
서버리스 함수	Lambda	Functions	Cloud Functions
관리형 K8s	EKS	AKS	GKE
컨테이너 레지스트리	ECR	ACR	Artifact Registry
모니터링	CloudWatch	Azure Monitor	Cloud Monitoring
IaC(자체)	CloudFormation	ARM/Bicep	Deployment Manager

제2장 점검

- ☐ IaaS·PaaS·SaaS에서 OS 패치 책임이 각각 누구에게 있는가?
- ☐ t3.micro에서 패밀리·세대·크기를 해석할 수 있는가?
- ☐ EBS·S3·EFS의 접근 방식 차이를 설명할 수 있는가?
- ☐ 퍼블릭/프라이빗 서브넷의 차이와 NAT의 역할을 설명할 수 있는가?
- ☐ Scale-out의 전제 조건(무상태 설계)을 이해했는가?

제3장

클라우드 생태계의 핵심 개념

SDN · 제로 트러스트 · API · MSA · DevOps/CI/CD — 전체 그림 그리기

3.0 이 장의 목적

서로 다른 분야에서 온 개념들이
어떻게 하나의 "클라우드 생태계"로 맞물리는지 지도를 그린다

- SDN(네트워크) · 제로 트러스트(보안) · API/MSA(아키텍처) · CI/CD(개발 방법론)
- 각 개념을 쉽게, 그러나 정확하게 → 마지막에 전체 연결

3.1 SDN — 왜 알아야 하는가

- **출발 질문** — 클라우드 환경에서 수만 명의 사용자가 동시에 가상 네트워크를 만드는데, 어떻게 장비 설정 충돌없이 실시간으로 제공이 될 수 있는가?
- **전통 방식** — 스위치·라우터에 케이블 연결, 장비마다 명령어 설정. 변경에 수 시간~수 일
(스위치와 라우터에는 제어 영역, 데이터 영역, 관리 영역이 있음)
- **답** — SDN(Software Defined Network). 클라우드 네트워크가 마법이 아니라 기술임을 이해하는 열쇠

SDN 핵심 — 제어와 전달의 분리

전통 네트워크

- 장비마다 두뇌(제어)+손발(전달) 일체
- 구성 변경 = 장비별 CLI 명령어를 직접 입력
- 자동화 제한적
- 휴먼에러 발생

SDN

- 두뇌를 중앙 SDN 컨트롤러로 분리
- 장비는 전달만, 판단은 소프트웨어가 일괄
- 구성 변경 = 소프트웨어 명령(수 초)
- API 기반 완전 자동화 → VPC 즉시 생성의 기반

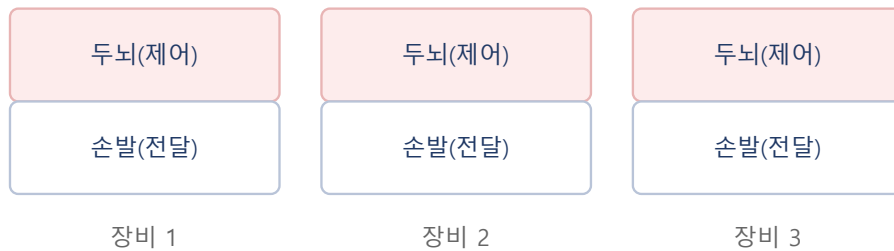
콘솔에서 VPC를 만들면 생기는 일

물리 케이블은 그대로 —
SDN 컨트롤러가 수많은 장비에 설정을 소프트웨어로 즉시 배포

- 논리적으로 격리된 나만의 네트워크가 몇 초 만에 생성
- 이것이 "소프트웨어로 정의된" 네트워크의 의미

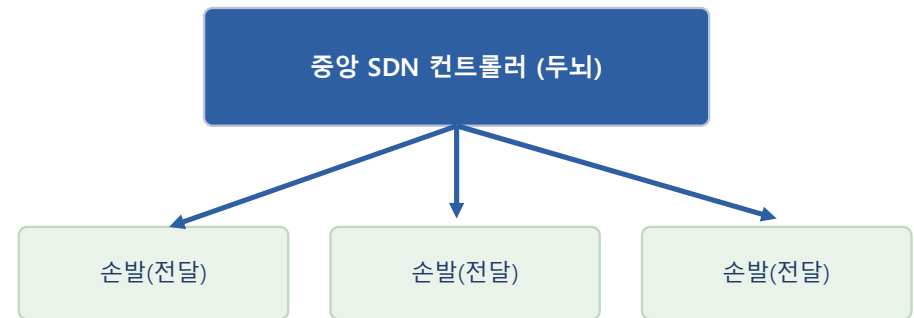
SDN을 그림으로 — 두뇌를 중앙으로

전통 네트워크



장비마다 두뇌 → 하나씩 수작업 설정

SDN



소프트웨어 명령 한 번 → 전체 일괄 반영

제어(판단)와 전달(운반)의 분리 — VPC가 클릭 몇 초 만에 생기는 기술적 이유

3.2 보안 패러다임 전환

경계 보안 (Perimeter)

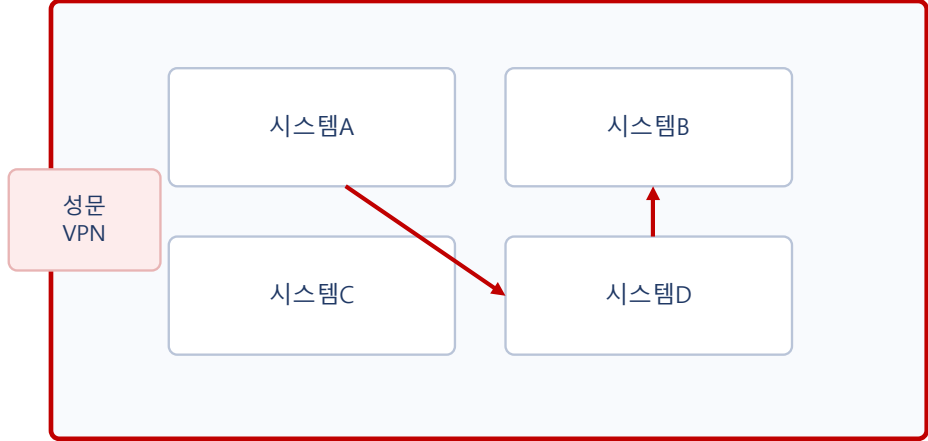
- 전제: 내부망=안전, 외부=위험
- 방화벽 성벽 + VPN 성문
- 성문 통과 후엔 내부 이동 자유
- 한계: 재택·클라우드로 경계 소멸, 침투 후 횡적 이동 (Lateral Movement) 무방비

제로 트러스트 (Zero Trust)

- 전제: 아무도 기본 신뢰하지 않음
- Never Trust, Always Verify
- 모든 접근을 매번 검증
- 내부망 위치는 신뢰의 근거가 아님

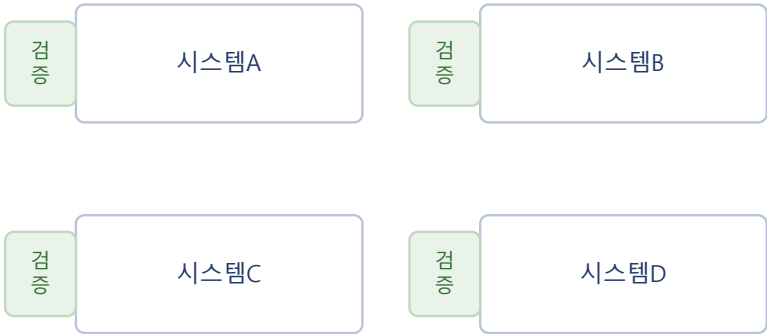
두 보안 모델을 그림으로

경계 보안 — 성벽 모델



성문만 통과하면 내부 이동 자유 — 침투 시 무방비

제로 트러스트 — 매번 검증



모든 접근마다 신원·권한·상태 확인 — 내부도 예외 없음

전제의 전환: "안은 안전하다" → "아무도 기본으로 믿지 않는다(Never Trust, Always Verify)"

제로 트러스트 — 모든 요청에 확인하는 것

- **누가** — 사용자 신원 확인. 비밀번호 + MFA(다중 인증)
- **어떤 기기**로 — 등록 기기 여부, 보안 패치 상태
- **무엇**에 — 그 자원에 대한 권한 여부. 딱 필요한 만큼만(최소 권한)
- **평소와 다른**가 — 위치·시간·행동 패턴 이상 시 추가 인증 또는 차단

제로 트러스트 실천 요소와 AWS 대응

요소	내용	AWS 대응
강력한 신원 확인	모든 사용자·시스템에 신원, MFA 의무	IAM 사용자·역할, MFA
최소 권한	필요 최소한만 부여	IAM 정책 좁게 작성
마이크로 세그멘테이션	네트워크를 잘게 나눠 구간 통제	서브넷 분리 + 보안그룹
지속적 검증	접근마다 확인	세션 관리, 임시 자격증명
전 구간 암호화	내부 통신도 암호화	S3/EBS 암호화, HTTPS/TLS
가시성·로깅	모든 접근 기록, 이상 탐지	CloudTrail, VPC Flow Logs

제로 트러스트는 제품이 아니라 원칙 — 2일차 보안그룹 최소 개방, 3일차 IAM 최소 권한이 모두 그 실천이다

3.3 API — 클라우드 세계의 공용어

- **정의** — 프로그램끼리 기능을 요청·응답하는 약속된 창구. 정해진 주소·형식으로 요청, 정해진 형식(JSON)으로 응답
- **예** — 날씨 앱은 기상 API에 "서울 날씨" 요청 → JSON 응답을 화면에 표시. 서로 내부 구조를 몰라도 협업
- **클라우드의 모든 기능은 API** — 콘솔 클릭도 내부적으로는 API 호출. Terraform = AWS API를 대신 호출하는 도구
- **자동화의 전제** — API가 있어야 스크립트 자동화 가능. MSA의 서비스 간 대화도 전부 API

REST API와 API Gateway

- **REST** — HTTP 동작을 그대로 활용: GET(조회)/POST(생성)/PUT(수정)/DELETE(삭제). 예: GET /users/10
- **JSON** — 요청·응답의 표준 데이터 형식. 사람이 읽을 수 있는 구조화 텍스트
- **API Gateway** — 여러 API 앞의 공동 정문. 인증·호출량 제한·라우팅·로깅을 한곳에서
- **AWS** — Amazon API Gateway가 이 역할. 서비스가 많아질수록 필수

3.4 모놀리식 vs MSA

모놀리식 (Monolithic)

- 전체 기능이 한 덩어리 프로그램
- 초기 개발 단순
- 한 줄 수정에도 전체 재배포
- 한 기능 장애가 전체 중단으로
- 부하 집중 기능만 확장 불가

MSA (Microservices)

- 기능 단위 독립 서비스로 분리
- 서비스별 독립 배포·확장
- 장애가 해당 서비스로 국지화
- 서비스별 기술 스택 선택 가능
- 서로는 API로 통신

MSA에 대한 균형 감각

**MSA는 무조건 정답이 아니다 —
"함께 배포하는 비용 > 나눠 관리하는 비용"이 될 때 가는 것**

- 서비스 분할만큼 배포·모니터링·통신 관리 복잡도 증가
- 소규모 시스템은 모놀리식이 오히려 합리적

MSA · 컨테이너 · 쿠버네티스의 관계

**MSA(설계) → 컨테이너(포장) → 쿠버네티스(운영)
한 세트로 움직인다**

- MSA로 배포 단위가 1개 → 수십 개로 증가
- 수십 개를 일관되게 포장 = Docker / 자동 배치·복구·확장 = K8s
- 오늘 오후, 이 맥락 위에서 컨테이너·K8s 개념 학습

3.5 DevOps — 개발과 운영의 벽 허물기

- **전통 구조의 문제** — Dev는 "빨리 배포", Ops는 "변경 최소화". 배포가 분기 1회 행사 → 변경 누적 → 위험 증가
- **DevOps** — 개발과 운영을 하나의 흐름·하나의 책임으로 통합하는 문화·방법론
- **핵심 원칙** — 작게, 자주, 자동으로 배포 → 변경당 위험 축소, 문제 조기 발견

CI/CD 파이프라인의 흐름

각 단계가 사람 손 없이 자동으로 이어지는 것이 핵심

1

개발자가 코드를 Git 저장소에 푸시

2

CI 도구가 감지 → 자동 빌드 + 테스트 (GitHub Actions, Jenkins 등)

3

통과 시 배포 패키지(컨테이너 이미지) 생성 → 레지스트리 저장

4

CD가 개발 → 스테이징 → 운영 환경으로 순차 배포

5

모니터링이 이상 감지 시 이전 버전으로 자동 롤백

CI / CD / IaC 정리

- **CI (지속적 통합)** — 코드 푸시마다 빌드·테스트 자동 실행. "몇 달치 코드를 한꺼번에 합치다 터지는" 사고 방지
- **CD (지속적 제공/배포)** — 통과한 코드를 배포 가능 상태로(Delivery), 운영까지 자동 배포(Deployment)
- **클라우드와의 궁합** — 테스트 환경을 API로 즉석 생성·폐기 가능
- **IaC** — 앱 배포 자동화가 CI/CD라면, 인프라 구성 자동화가 IaC. 코드=문서, Git으로 이력 관리. 3일차 Terraform 실습

3.6 전체 지도 — 개념의 연결

클라우드 생태계의 판

1

물리 기반 — 데이터센터 서버들이 스파인-리프 네트워크로 연결

2

가상화 + SDN — 하이퍼바이저가 VM을, SDN이 고객별 VPC를 생성 (여기까지 CSP 영역)

3

사용자 인프라 — API(콘솔·CLI·Terraform)로 프로비저닝, IaC로 코드화

4

애플리케이션 — MSA로 분할, API로 통신, 컨테이너로 포장, K8s로 운영

5

배포·운영 — CI/CD 파이프라인으로 자동 빌드·테스트·배포 (문화 전체가 DevOps)

6

보안 관통 — 전 계층에 제로 트러스트 원칙 적용

전체 지도를 한 장의 그림으로



아래에서 위로 쌓이고, 보안이 옆에서 전부를 관통한다 — 3일 교육이 이 그림을 한 바퀴 돈다

제3장 점검

- ☐ SDN의 제어/전달 분리가 무슨 의미인지 설명할 수 있는가?
- ☐ 스파인-리프에서 서버 간 통신이 항상 2홉인 이유는?
- ☐ 경계 보안과 제로 트러스트의 전제 차이를 한 문장으로?
- ☐ MSA에서 서비스 간 통신 수단은 무엇인가?
- ☐ CI와 CD가 각각 무엇을 자동화하는가?
- ☐ 전체 지도에서 Terraform과 K8s는 각각 어느 단계인가?

제4장

클라우드 트렌드와 가치·이슈

시장 동향 · 클라우드 네이티브 · FinOps · 도입 전략

4.1 시장 구조와 성장 동인

- **과점 구조** — AWS·Azure·GCP 3사가 60% 이상 점유, 연평균 두 자릿수 성장
- **3사 포지션** — AWS(1위·최다 서비스), Azure(MS 통합·엔터프라이즈), GCP(K8s·AI 원조)
- **성장 동인** — 생성형 AI의 GPU 수요, 레거시 현대화, 공공 클라우드 전환 정책, 금융 규제 완화
- **국내 이원화** — 민간(글로벌 CSP) vs 공공·금융(국내 CSP) + MSP 위탁 보편화

4.2 클라우드 네이티브의 4요소

"클라우드에서 돌아간다"가 아니라 "클라우드답게 만들어졌다"

- **컨테이너** — 실행 환경째 패키징, 어디서나 동일 실행
- **MSA** — 독립 배포 가능한 서비스 분할 (3.4절)
- **선언적 API·자동화** — "서버 3대 유지하라" 선언 → 시스템이 상태를 맞춤. K8s·Terraform의 철학
- **불변 인프라** — 운영 서버를 고치지 않고 새 서버로 교체. 구성 편차 원천 차단

그 외 주요 트렌드

- **서버리스 확산** — FaaS+관리형 조합이 이벤트 기반 워크로드 표준
- **플랫폼 엔지니어링** — 사내 개발자 플랫폼(IDP) 구축. DevOps의 다음 단계
- **FinOps** — 비용 가시화→최적화→거버넌스 사이클을 조직적으로 운영
- **엣지 컴퓨팅** — 밀리초 응답 워크로드를 사용자 인접 지점에서 처리
- **지속가능성** — PUE·탄소 배출이 CSP 평가 항목으로

4.3 도입 이슈 ① 보안·컴플라이언스 / ② 비용

- **최대 리스크 = 설정 오류** — 스토리지 공개, 과도한 권한, 키 유출. 공동 책임 모델의 조직적 이해가 최우선
- **규제 선행 검토** — 공공 CSAP, 금융 감독규정, 개인정보 위탁·국외이전
- **FinOps 실무 4종** — 태깅 의무화(가시화), 예산 알림, 최적화 루틴(정리·사이징·약정·스팟), 아키텍처 차원 절감(CDN·수명 주기)

도입 이슈 ③ 벤더 종속 / ④ 조직 전환

- **종속의 3요소** — 데이터 이전 비용, 전용 서비스 의존 코드, 인력의 특정 CSP 숙련
- **완화 전략** — 컨테이너로 실행 환경 표준화, Terraform으로 인프라 정의 중립화. 목표는 "이전 가능성 유지"
- **조직 전환** — 역량 전환 교육, 결재 프로세스 → 셀프서비스+거버넌스, 운영 모델(자체/MSP/혼합) 결정

4.4 클라우드 전환 전략 — 6R

전략	내용	속도/효과
Rehost (Lift & Shift)	수정 없이 인프라만 이전	속도 최고 / 효과 최소
Replatform	DB를 RDS로 등 부분 최적화 이전	균형
Refactor	MSA·서버리스로 재설계	비용 최대 / 효과 최대
Repurchase	자체 시스템을 SaaS로 대체	구축 제거
Retain	규제·기술 제약으로 잔류	현상 유지
Retire	저활용 시스템 폐기	비용 절감

4.5 산업별 활용과 직무 지형

- **산업별** — OTT(오토스케일+CDN), 이커머스(순간 트래픽 흡수), 금융(채널부터 전환·계정계 하이브리드), 공공(CSAP 기반 재구축), 게임(탄력성=생존)
- **직무** — 아키텍트(설계), 클라우드 엔지니어(구축·자동화), DevOps(CI/CD·컨테이너), SRE(안정성·모니터링), 보안, FinOps
- **본 과정 연결** — 2일차 실습=엔지니어 영역, 3일차 Terraform=자동화, 보안·모니터링=SRE·보안 영역

오전 세션 요약

- 클라우드 = 자원 풀 + 가상화 + 자동화 제어 + 계측·과금. 소유가 아닌 사용, 셀프서비스, 종량제
- 도입 판단은 이분법이 아닌 워크로드별 TCO 비교. 부하 변동이 클수록 유리
- 서비스 모델(IaaS/PaaS/SaaS)은 관리 책임의 경계, 배포 모델은 통제와 민첩성의 트레이드오프
- 리전—AZ—엣지 / Multi-AZ가 고가용성 기본 공식. EC2·S3·VPC 상세는 내일 실습의 배경지식
- 물리망=스파인-리프, 논리망=SDN / 보안은 경계 방어 → 제로 트러스트로